

0.90.1. L'écran titre — START, un autre écran

Le titre du 0.90, et START qui le fait disparaître : l'écran se vide, un autre apparaît. Une variable retient sur quel écran on est.

1. Ce qu'il faut comprendre

C'est le 0.90, avec une seule chose en plus : quand on appuie sur **START**, le titre s'en va et un **autre écran** apparaît (ici, « C EST PARTI »).

`bouton(START)` rend 1 tant que START est enfoncé. On le regarde à **chaque image**, dans la boucle.

Ce qui est nouveau ici : une variable qui dit sur quel écran on est. `ecranTitre` vaut **1** tant qu'on est sur le titre, **0** ensuite. Pourquoi ? Un appui sur START dure plusieurs images. Sans cette variable, l'écran serait vidé et réécrit à **chaque image** où le doigt reste sur le bouton. Avec elle, le changement se fait **une seule fois** : dès la première image, `ecranTitre` passe à 0, et la condition `ecranTitre == 1 && bouton(START)` n'est plus jamais vraie.

`&&` veut dire « et » : il faut les deux à la fois, être sur le titre **et** appuyer sur START.

Vider l'écran : il n'y a pas de fonction qui efface tout d'un coup. On le fait ligne par ligne : une boucle `for` sur les **18 lignes** (0 à 17), et sur chacune `effacer(0, y, 20)` efface **20 cases** à partir de la colonne 0, toute la largeur.

La couleur reste : `couleurTexte` a réglé l'encre de toutes les lettres ; le nouvel écran est orange aussi.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <couleurTexte>` → [0.87.7. La fonction couleurTexte\(\) — la couleur des lettres](#)
- `#include <texteGrand>` → [0.86.3. La fonction texteGrand\(\) — un texte agrandi](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1444 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage          59 o ← la console, à l'allumage - la mise en route de la console
texte_grand_0()    195 o ← main() - ta fonction
main()             102 o ← démarrage - ta fonction
VBlank              8 o  ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage      28 o ← image() - attend l'image suivante
AttendreVBlank     16 o ← démarrage - attend que l'écran soit libre
AttendreAcces      12 o ← EffacerCases, EcrireTexte - attend de pouvoir écrire une case
AdresseCase        10 o ← poser(), effacer() - trouve l'adresse d'une case de l'écran
EffacerCases       12 o ← effacer() - efface des cases
```

EcrireTexte	10 o	← texte()	– écrit un texte, lettre par lettre
CopierTuiles	18 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
LireManette	37 o	← bouton()	– lit les huit boutons
Tuiles	848 o	← CopierTuiles	– la police entière, et 9 tuiles dessinées
"APPUIE SUR START"	16 o	← texte()	– un texte gravé
"C EST PARTI"	11 o	← texte()	– un texte gravé
GRAND_TABLE_0_0_0	12 o	← poser()	– une table gravée
GRAND_TABLE_0_1_1	12 o	← poser()	– une table gravée
GRAND_TABLE_0_2_2	12 o	← poser()	– une table gravée
GRAND_TABLE_0_3_3	12 o	← poser()	– une table gravée

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 4	122 o sur place, dans texte_grand_0() + AdresseCase (10 o), GRAND_TABLE_0_0_0 (12 o),
GRAND_TABLE_0_1_1 (12 o), GRAND_TABLE_0_2_2 (12 o), GRAND_TABLE_0_3_3 (12 o)		
texte()	× 2	22 o sur place, dans main() + EcrireTexte (10 o)
effacer()	× 1	14 o sur place, dans main() + AdresseCase (10 o), EffacerCases (12 o)
couleurFond()	× 1	12 o sur place, dans main()
bouton()	× 1	5 o sur place, dans main() + LireManette (37 o)
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)

Tes fonctions appelées :

texte_grand_0() × 1	3 o sur place, dans main()
---------------------	----------------------------

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <couleurTexte> // choisit la couleur des lettres
#include <texteGrand>   // écrit un texte agrandi, de 1 à 20 fois
#include <texte>         // écrit un texte à l'écran
#include <bouton>        // lit un bouton de la manette
#include <effacer>       // efface des cases, ou tout le fond
```

2. UNE VARIABLE — ECRANTITRE

```
uint8_t ecranTitre = 1; // sur quel écran on est : 1 = le titre, 0 = l'écran d'après
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // Le titre du 0.90 :
    couleurTexte(31, 16, 0);           // de l'orange
    texteGrand(4, 5, "JEU", 4);        // le grand titre
    texte(2, 12, "APPUIE SUR START"); // la consigne

    while (true) {
        image();

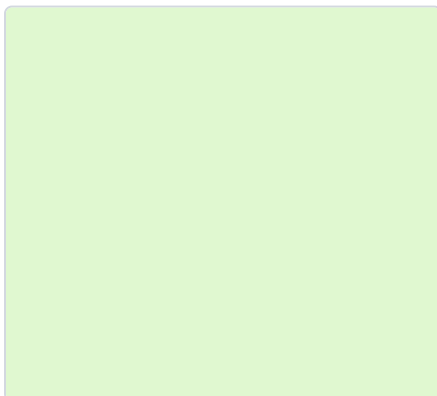
        // On change d'écran si on est SUR LE TITRE ET que START est appuyé.
        //
        //   ecranTitre == 1    &&    bouton(START)
        //   |                  |
        //   |                  |   +-- START est enfoncé (1), ou pas (0)
        //   |                  |   +----- « et » : il faut les deux à la fois
        //   |                  +----- on est encore sur le titre
        if (ecranTitre == 1 && bouton(START)) {
            ecranTitre = 0;           // tout de suite : ce bloc ne se refera plus jamais,
                                     // même si le doigt reste sur START

            // Vider l'écran : 18 lignes (0 à 17), 20 cases chacune.
            //   y = 0 : effacer(0, 0, 20)   la ligne du haut
            //   y = 1 : effacer(0, 1, 20)   celle d'en dessous
            //   ...
            //   y = 17 : effacer(0, 17, 20) la ligne du bas
            for (uint8_t y = 0; y < 18; y++) {
                effacer(0, y, 20);
            }

            texte(5, 8, "C EST PARTI"); // le nouvel écran
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



C EST PARTI

START — l'écran a changé

5. Ce qu'on doit voir

Le grand JEU orange et APPUIE SUR START ; appuie sur START (touche Entrée) : l'écran se vide et C EST PARTI apparaît.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ avant START : le titre et sa consigne
- ✓ après START : la consigne et le grand JEU ont disparu (0 cases du titre restent)
- ✓ le nouvel écran : C EST PARTI
- ✓ `ecranTitre` est passé à 0

Ensuite : **0.90.2. L'écran titre — une lettre qui file** — chapitre 0 : Avant tout. · [gameboy3](#) — écrire une cartouche Game Boy en C++